



An ACCOMPLISH Project Module

**The Shooting Method for Linear Boundary-Value
Problems**

Prof. Bo Dong

University of Massachusetts Dartmouth

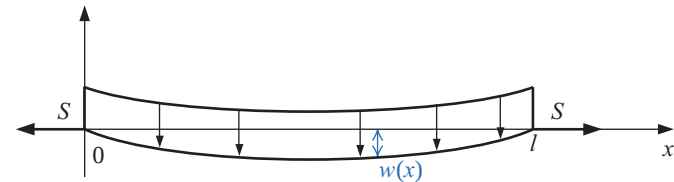
The second-order boundary-value problems (BVP):

$$\begin{aligned}y'' &= f(x, y, y'), \quad a \leq x \leq b, \\y(a) &= \alpha, \quad y(b) = \beta.\end{aligned}$$

Example: Deflection of a beam subject to uniform loading with fixed ends:

$$\begin{aligned}w''(x) &= \frac{S}{EI}w(x) + \frac{q}{2EI}x(x-l), \quad 0 \leq x \leq l, \\w(0) &= w(l) = 0.\end{aligned}$$

Here, $w(x)$ is the deflection distance.
 l is the length of the beam.
 q is the intensity of the uniform load.
 E is the modulus of elasticity.
 S is the stress at the endpoints.
 I is the central moment of inertia.



Definition: If $f(x, y, y')$ has the form

$$f(x, y, y') = p(x)y' + q(x)y + r(x),$$

then the differential equation $y'' = f(x, y, y')$ is **linear**.

Theorem: (Existence and uniqueness of the solution to linear BVP):
If the linear BVP

$$\begin{aligned} y'' &= p(x)y' + q(x)y + r(x), \quad a \leq x \leq b, \\ y(a) &= \alpha, \quad y(b) = \beta, \end{aligned}$$

satisfies

(i) $p(x)$, $q(x)$, and $r(x)$ are continuous on $[a, b]$,

(ii) $q(x) > 0$ on $[a, b]$,

then the problem has a unique solution.

The Shooting Method for solving linear BVP

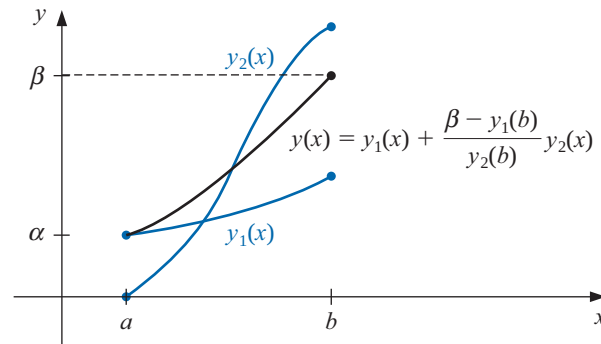
Idea: Decompose a linear BVP

$$(BVP) : \begin{cases} y'' = p(x)y' + q(x)y + r(x), & a \leq x \leq b, \\ y(a) = \alpha, & y(b) = \beta, \end{cases}$$

into two IVPs:

$$(IVP\ 1) : \begin{cases} y'' = p(x)y' + q(x)y + r(x), & a \leq x \leq b, \\ y(a) = \alpha, & y'(a) = 0, \end{cases}$$

$$(IVP\ 2) : \begin{cases} y'' = p(x)y' + q(x)y, & a \leq x \leq b, \\ y(a) = 0, & y'(a) = 1. \end{cases}$$



Suppose $y_1(x)$ is the solution to (IVP 1) and $y_2(x)$ is the solution to (IVP 2). Then the solution to (BVP) is

$$y(x) = y_1(x) + \frac{\beta - y_1(b)}{y_2(b)} y_2(x).$$

Proof:

(i) It is easy to check that $y = y_1 + c y_2$ satisfies $y'' = p(x)y' + q(x)y + r(x)$ and $y(a) = \alpha$ for any constant c .

(ii) If $y = y_1 + c y_2$ satisfies $y(b) = \beta$, then $c = \frac{\beta - y_1(b)}{y_2(b)}$.

Numerical methods for solving a linear BVP:

Step 1: solve (IVP 1) by some numerical method (e.g. Euler or RK4).

Step 2: solve (IVP 2) by some numerical method (e.g. Euler or RK4).

Step 3: use the linear combination

$$y(x) = y_1(x) + \frac{\beta - y_1(b)}{y_2(b)} y_2(x)$$

to get the approximate solution to the (BVP).

Example: Solve the BVP

$$\begin{cases} y'' = -\frac{2}{x}y' + \frac{2}{x^2}y + \frac{\sin(\ln x)}{x^2}, & 1 \leq x \leq 2, \\ y(1) = 1, \quad y(2) = 2, \end{cases}$$

by using the linear shooting method with $h = 0.1$ and compare the results to those of the exact solution

$$y = 1.1392070132x - \frac{0.03920701320}{x^2} - \frac{3}{10}\sin(\ln x) - \frac{1}{10}\cos(\ln x).$$

Solution: First, we rewrite the given BVP into the following two IVPs:

$$(IVP\ 1) : \begin{cases} y'' = -\frac{2}{x}y' + \frac{2}{x^2}y + \frac{\sin(\ln x)}{x^2}, & 1 \leq x \leq 2, \\ y(1) = 1, \quad y'(1) = 0, \end{cases}$$

$$(IVP\ 2) : \begin{cases} y'' = -\frac{2}{x}y' + \frac{2}{x^2}y, & 1 \leq x \leq 2, \\ y(1) = 0, \quad y'(1) = 1. \end{cases}$$

Next, we rewrite each IVP as a system of first-order equations so that we can apply numerical solvers for IVPs.

- For (IVP 1), let $y_1 = y, y_2 = y'$ and we get the system

$$\begin{aligned}y_1' &= y_2, \\y_2' &= -\frac{2}{x}y_2 + \frac{2}{x^2}y_1 + \frac{\sin(\ln x)}{x^2}, \\y_1(1) &= 1, y_2(1) = 0.\end{aligned}$$

- For (IVP 2), let $y_1 = y, y_2 = y'$ and we get the system

$$\begin{aligned}y_1' &= y_2, \\y_2' &= -\frac{2}{x}y_2 + \frac{2}{x^2}y_1, \\y_1(1) &= 0, y_2(1) = 1.\end{aligned}$$

Finally, we use the fourth-order Runge-Kutta method to solve the initial value problems above and combine their solutions to get that for the BVP (see the MATLAB code and results on the next two pages).

MATLAB code:

```
function [x,w]=LinearShooting(a,b,alpha,beta,h,f1,f2)
    [x,u]=RK4(a,b, [alpha,0],h,f1);
    [~,v]=RK4(a,b,[0,1],h,f2);
    w=u(:,1)+(beta-u(end,1))/v(end,1)*v(:,1);
end
```

```
function [t,w]=RK4(a,b,alpha,h,f)
    N=ceil((b-a)/h);
    t(1)=a;
    w(1,:)=alpha;
    for i=1:N
        K1=h*f(t(i),w(i,:));
        K2=h*f(t(i)+h/2,w(i,:)+K1/2);
        K3=h*f(t(i)+h/2,w(i,:)+K2/2);
        K4=h*f(t(i)+h,w(i,:)+K3);
        w(i+1,:)=w(i,:)+(K1+2*K2+2*K3+K4)/6;
        t(i+1)=t(i)+h;
    end
end
```


MATLAB code:

```
function example
    f1=@(x,y) [y(2), -2/x*y(2)+2/x^2*y(1)+sin(log(x))/x^2];
    f2=@(x,y) [y(2), -2/x*y(2)+2/x^2*y(1)];

    a=1;  b=2;  alpha=1;  beta=2;  h=0.1;

    [x,w]=LinearShooting(a,b,alpha,beta,h,f1,f2);

    y=1.1392070132*x-0.03920701320./x.^2-3/10*sin(log(x))-1/10*cos(log(x));

    disp(' ')
    disp('    x(i)      w(i)      y(i)      |y(i)-w(i)| ')
    disp(' -----')
    for i=1:length(x)
        fprintf('    %5.2f    %7.4f    %7.4f    %0.5g\n', ...
                x(i), w(i), y(i),abs(w(i)-y(i)))
    end
    disp(' ')

end
```

MATLAB Outputs:

```
>> example
```

x(i)	w(i)	y(i)	y(i)-w(i)

1.00	1.0000	1.0000	0
1.10	1.0926	1.0926	1.3436e-07
1.20	1.1871	1.1871	1.3369e-07
1.30	1.2834	1.2834	9.7824e-08
1.40	1.3814	1.3814	6.0199e-08
1.50	1.4812	1.4812	3.0676e-08
1.60	1.5824	1.5824	1.0818e-08
1.70	1.6850	1.6850	4.8973e-10
1.80	1.7889	1.7889	4.9909e-09
1.90	1.8939	1.8939	4.3460e-09
2.00	2.0000	2.0000	6.9513e-11